

CV32E40P Front-End Script 분석 보고서

문서 목적

CV32E40P 프로젝트의 front-end implementation script를 학습 관점에서 해석한다. 본 보고서는 디자인 자료가 아니라 기술 보고서 형식으로, flow 순서·script 역할·주요 명령·제약 조건·후속 tool 연계를 설명한다.

분석 대상

구분	대상
Project	CV32E40P RISC-V Core Front-End Flow
Main Tool	Synopsys Design Compiler, Formality, DFT Compiler, TetraMAX, PrimeTime
Main Scope	Synthesis → FM R2N → DFT → FM N2N → ATPG → STA
기준 Mode	Functional 10ns baseline + scan insertion flow

읽는 방법

- 1. 먼저 전체 flow를 본다.
- 2. library/filelist/SDC를 먼저 이해한다.
- 3. DC가 만든 artifact가 Formality/DFT/PrimeTime으로 어떻게 전달되는지 본다.
- 4. 각 script는 “무엇을 읽고, 무엇을 만들고, 다음 단계에서 어떻게 쓰이는가” 기준으로 읽는다.

목적: CV32E40P front-end script를 공부용으로 읽기 쉽게 정리한다. 각 script의 역할, 주요 명령의 의미, 왜 필요한지, 후속 tool과 어떻게 연결되는지를 기준으로 본다.

0. 전체 Flow

```

RTL / wrapper / filelist
↓
DC: analyze → elaborate → link smoke test
↓
DC Topographical: SDC 적용 → compile_ultra -spg -gate_clock
↓
pre-DFT netlist / DDC / SDC / SDF / SVF
↓
Formality R2N: RTL ↔ synthesized netlist
↓
DFT Compiler: scan protocol → DFT DRC → insert_dft
↓
post-DFT netlist / DDC / SDC / SDF / SPF / SVF
↓
Formality N2N: pre-DFT netlist ↔ post-DFT netlist
↓
TetraMAX: scan DRC / stuck-at ATPG
↓
PrimeTime: pre/post-DFT SDF STA

```

핵심은 **DC** 결과를 그대로 믿지 않고, **FM / DFT / ATPG / PT**가 다시 검증하게 만드는 구조다.

1. 디렉터리 역할

경로	역할
configs/	공통 library setup
filelists/	DC/Formality가 읽을 RTL 목록
constraints/	SDC timing constraint
2_Synthesis/	DC synthesis
2.5_FM_R2N/	Formality RTL-to-netlist
3_DFT/	DFT Compiler scan insertion
4_ATPG/	TetraMAX ATPG
5_FM_N2N/	Formality netlist-to-netlist
6_STA/	PrimeTime STA
00_Project_Tracking/	실행 결과/결정/상태 요약

2.5_FM_R2N 이 DFT 앞에 있는 이유는 간단하다. DFT를 넣기 전에 **pre-DFT netlist**가 RTL과 같은지 먼저 확인해야 한다.

2. Library Setup

파일:

```
configs/library_setup.tcl
```

핵심 코드:

```
set SAED32_R00T /DATA/home/edu135/lib/SAED32_EDK

set RVT_TT_DB $SAED32_R00T/lib/stdcell_rvt/db_nldm/saed32rvt_tt1p05v25c.db
set LVT_TT_DB $SAED32_R00T/lib/stdcell_lvt/db_nldm/saed32lvt_tt1p05v25c.db
set HVT_TT_DB $SAED32_R00T/lib/stdcell_hvt/db_nldm/saed32hvt_tt1p05v25c.db

set_app_var target_library [list $RVT_TT_DB $LVT_TT_DB $HVT_TT_DB]
set_app_var link_library [list * $RVT_TT_DB $LVT_TT_DB $HVT_TT_DB]
```

줄	의미	필요한 이유
SAED32_R00T	SAED32 library root path	모든 library path의 기준점
RVT_TT_DB	regular-Vt TT timing DB	기본 standard cell mapping 후보
LVT_TT_DB	low-Vt TT timing DB	빠른 cell 후보. timing에 유리하지만 leakage 증가 가능
HVT_TT_DB	high-Vt TT timing DB	느리지만 leakage 절감 가능
target_library	DC가 mapping할 cell library	RTL operator를 실제 cell로 바꾸는 대상
link_library	reference resolve용 library	submodule/stdcell reference를 연결하기 위함

주의:

```
target_library = 어떤 cell로 합성할 것인가
link_library   = design 안 reference를 어디서 찾을 것인가
```

`link_library [list * ...]`의 `*`는 현재 메모리에 읽힌 design도 link 대상으로 쓰겠다는 의미다.

3. RTL Filelist

파일:

```
filelists/cv32e40p_dc.tcl
```

구조:

```
set RTL_INC_DIRS [list \  
    rtl/cv32e40p/rtl/include \  
]  
  
set RTL_FILES [list \  
    ... package files ...  
    rtl/tech/cv32e40p_clock_gate.sv \  
    ... core modules ...  
    rtl/cv32e40p/rtl/cv32e40p_top.sv \  
    rtl/wrappers/cv32e40p_synth_wrap.sv \  
]
```

SystemVerilog는 compile order가 중요하다.

```
package → tech wrapper → lower modules → top → synth wrapper
```

순서가 틀리면 package type, module reference, clock gate wrapper가 unresolved로 깨진다.

항목	의미
RTL_INC_DIRS	include/package import 경로
cv32e40p_*_pkg.sv	typedef/enum/parameter package. 먼저 읽어야 함
cv32e40p_clock_gate.sv	synthesis/DFT/FM에서 필요한 clock-gating wrapper
cv32e40p_top.sv	원본 CPU IP top
cv32e40p_synth_wrap.sv	implementation flow용 wrapper top

wrapper를 쓰는 이유:

```
cv32e40p_top          = 원본 IP integration top  
cv32e40p_synth_wrap = synthesis/DFT/STA에 맞춘 implementation boundary
```

4. SDC Constraint

파일:

```
constraints/cv32e40p_func_10ns.sdc
```

핵심:

```
create_clock -name clk_i -period 10.0 [get_ports clk_i]
set_clock_uncertainty 0.1 [get_clocks clk_i]

set_case_analysis 0 [get_ports scan_cg_en_i]
set_case_analysis 0 [get_ports scan_en]
set_case_analysis 0 [get_ports scan_in]

set_false_path -from [get_ports rst_ni]

set_input_delay 1.0 -clock clk_i [get_ports {...}]
set_output_delay 1.0 -clock clk_i [get_ports {...}]
```

명령	의미	필요한 이유
<code>create_clock</code>	<code>clk_i</code> 를 10ns clock으로 정의	STA의 기준. 없으면 reg-to-reg timing 계산 불가
<code>set_clock_uncertainty 0.1</code>	clock margin 0.1ns	ideal clock만 보면 과하게 낙관적이라 margin 부여
<code>set_case_analysis 0</code> <code>scan_cg_en_i</code>	functional mode에서 scan clock-gating disable	scan/test path가 functional STA를 오염하지 않게 함
<code>set_case_analysis 0</code> <code>scan_en</code>	functional mode에서 scan shift disable	scan mux path 제외
<code>set_case_analysis 0</code> <code>scan_in</code>	scan input constant 처리	floating/unknown 방지
<code>set_false_path -from</code> <code>rst_ni</code>	async reset data timing 제외	reset은 별도 reset protocol/recovery/removal 문제
<code>set_input_delay 1.0</code>	외부 launching logic budget 가정	input-to-reg path budget 설정
<code>set_output_delay 1.0</code>	외부 capturing logic budget 가정	reg-to-output path budget 설정

이 SDC는 실제 SoC signoff constraint가 아니라 **functional 10ns baseline constraint**다.

5. Analyze / Elaborate / Link

파일:

```
2_Synthesis/0_Script/run_analyze_elab_link.tcl
```

목적:

합성 최적화 전에 RTL을 tool이 제대로 읽고 hierarchy를 만들 수 있는지 확인한다.

핵심 흐름:

```
source configs/library_setup.tcl
file mkdir ...
define_design_lib WORK -path 2_Synthesis/work
source filelists/cv32e40p_dc.tcl
set_app_var search_path [concat $search_path $RTL_INC_DIRS]
analyze -format sverilog $RTL_FILES
elaborate cv32e40p_synth_wrap
current_design cv32e40p_synth_wrap
link
check_design > ...
write -format ddc -hierarchy -output ...elab.ddc
```

명령	의미
<code>define_design_lib WORK</code>	analyze/elaborate 결과를 저장할 design library 생성
<code>analyze</code>	SystemVerilog source를 parse/compile
<code>elaborate</code>	parameter/hierarchy를 풀어 generic design 생성
<code>current_design</code>	이후 명령 대상 top 지정
<code>link</code>	module/stdcell reference 연결
<code>check_design</code>	unresolved, multi-driver, latch 등 기본 문제 확인
<code>write ddc</code>	elaboration 결과 저장

이 단계를 따로 두는 이유:

바로 compile하면 실패 원인이 RTL/filelist/library/constraint/optimization 중 어디인지 모호해진다.

6. DC Topographical Synthesis

파일:

```
2_Synthesis/0_Script/run_compile_10ns_topo.tcl
```

목적:

Milkyway/TLU+ 물리 정보를 반영해 placement-aware synthesis를 수행한다.

일반 compile과 차이:

```
run_compile_10ns.tcl      = wire-load 기반 일반 DC compile
run_compile_10ns_topo.tcl = Milkyway/TLU+ 기반 topographical compile
```

중요 physical collateral:

변수	의미
TECH_FILE	layer/via/design rule 등이 들어간 Milkyway technology file
TLUPLUS_MAX	worst RC extraction table
TLUPLUS_MIN	best RC extraction table
TLUPLUS_MAP	tech file layer와 ITF/TLU+ layer mapping
MW_RVT/LVT/HVT	Milkyway reference library
MW_DESIGN_LIB	현재 design용 Milkyway library

핵심 명령:

```

create_mw_lib -technology $TECH_FILE \
  -mw_reference_library [list $MW_RVT $MW_LVT $MW_HVT] \
  -open $MW_DESIGN_LIB

set_tlu_plus_files ...
check_tlu_plus_files
check_library

read_sdc constraints/cv32e40p_func_10ns.sdc
set_svf ...pre_dft_topo.svf
compile_ultra -spg -gate_clock
set_svf -off

```

명령	의미
<code>create_mw_lib</code>	physical-aware compile을 위한 MW design lib 생성
<code>set_tlu_plus_files</code>	RC 추정 table 설정
<code>check_tlu_plus_files</code>	TLU+ 설정 검증
<code>check_library</code>	library consistency 점검
<code>read_sdc</code>	10ns functional constraint 적용
<code>set_svf</code>	Formality용 optimization guide 기록 시작
<code>compile_ultra -spg -gate_clock</code>	physical guidance + clock gating 허용 합성

Output:

파일	후속 사용처
<code>pre_dft_topo.ddc</code>	DFT Compiler input
<code>pre_dft_topo.vg</code>	Formality/PrimeTime input
<code>pre_dft_topo.sdc</code>	STA/ICC2 constraint handoff
<code>pre_dft_topo.sdf</code>	SDF timing annotation
<code>pre_dft_topo.svf</code>	Formality R2N guide

7. Formality R2N

파일:

2.5_FM_R2N/0_Script/run_fm_r2n_topo.tcl

R2N:

Reference = RTL
Implementation = synthesized netlist

목적:

합성 후 netlist가 functional mode에서 RTL과 같은지 확인한다.

핵심:

```
set_svf $SVF_FILE
set_verification_clock_gate_reverse_gating true
read_db -technology_library [list $RVT_TT_DB $LVT_TT_DB $HVT_TT_DB]
read_sverilog -r -l2 -libname WORK $RTL_FILES
set_top r:/WORK/$TOP_NAME
set_constant -type port r:/WORK/$TOP_NAME/scan_cg_en_i 0
set_constant -type port r:/WORK/$TOP_NAME/scan_en 0
set_constant -type port r:/WORK/$TOP_NAME/scan_in 0
read_verilog -i -netlist -libname WORK $NETLIST
set_top i:/WORK/$TOP_NAME
match
verify
```

명령	의미
set_svf	DC optimization 정보를 FM에 제공
verification_clock_gate_reverse_gating	clock-gating equivalence 처리를 돕는 설정
read_sverilog -r	RTL을 reference side로 읽음
read_verilog -i	gate netlist를 implementation side로 읽음
set_constant scan_* 0	functional mode equivalence 조건 고정
match	compare point 대응
verify	equivalence 증명

scan port를 0으로 고정하는 이유:

검증하려는 것은 normal functional mode다. scan mode까지 섞으면 비교 의도가 흐려진다.

8. DFT Insertion

파일:

```
3_DFT/0_Script/run_insert_dft_10ns_topo.tcl
```

목적:

pre-DFT DDC를 읽어 muxed scan chain을 삽입하고 post-DFT artifact를 만든다.

중요 입력:

```
set PRE_DFT_DDC 2_Synthesis/2_Output/pre_dft_topo/cv32e40p_synth_wrap.pre_dft_topo.ddc
```

DDC를 쓰는 이유:

Verilog netlist보다 Synopsys 내부 정보가 많아 DFT Compiler reload에 유리하다.

scan 설정:

```
set_scan_configuration \  
  -style multiplexed_flip_flop \  
  -chain_count 1 \  
  -clock_mixing no_mix \  
  -add_lockup true  
  
set_dft_configuration \  
  -scan enable \  
  -connect_clock_gating enable
```

option	의미
<code>multiplexed_flip_flop</code>	functional DFF 앞에 scan mux를 붙이는 일반 scan style
<code>chain_count 1</code>	scan chain 1개 구성
<code>clock_mixing no_mix</code>	clock domain을 한 chain에 섞지 않음
<code>add_lockup true</code>	clock skew/edge 차이에 대비한 lockup latch 허용
<code>connect_clock_gating enable</code>	clock-gating test enable 연결 처리

DFT signal:

```
set_dft_signal -view existing_dft -type ScanClock -port clk_i -timing {45 55}
set_dft_signal -view existing_dft -type Reset -port rst_ni -active_state 0
set_dft_signal -view existing_dft -type TestMode -port scan_cg_en_i -active_state 1

set_dft_signal -view spec -type ScanEnable -port scan_en -active_state 1
set_dft_signal -view spec -type ScanDataIn -port scan_in
set_dft_signal -view spec -type ScanDataOut -port scan_out
set_scan_path chain0 -view spec -scan_data_in scan_in -scan_data_out scan_out
```

existing_dft 와 spec 차이:

```
existing_dft = 현재 design에 이미 존재하는 test control을 tool에 알려줌
spec        = 새로 만들 scan 구조를 지정함
```

실행:

```
create_test_protocol
dft_drc
set_svf ...post_dft_topo.svf
insert_dft
set_svf -off
```

명령	의미
<code>create_test_protocol</code>	scan clock/reset/test signal 기반 protocol 생성
<code>dft_drc</code>	insert 전 scan 가능성 검사
<code>insert_dft</code>	실제 scan mux/chain 삽입
<code>set_svf</code>	N2N Formality용 DFT 변환 정보 기록

Output:

파일	후속 사용처
<code>.spf</code>	TetraMAX scan DRC/ATPG
<code>.vg</code>	FM N2N, PT STA, ATPG
<code>.ddc</code>	Synopsys reload
<code>.sdc</code>	STA constraint handoff
<code>.sdf</code>	delay annotation
<code>.svf</code>	FM N2N guide

9. Formality N2N

파일:

```
5_FM_N2N/0_Script/run_fm_n2n_topo.tcl
```

N2N:

```
Reference      = pre-DFT synthesis netlist
Implementation = post-DFT scan netlist
```

목적:

scan insertion 이후에도 functional mode 동작이 유지되는지 확인한다.

검증 chain:

```
RTL == pre-DFT netlist == post-DFT netlist(functional mode)
```

핵심:

```

set_svf $SVF_FILE
read_verilog -r -netlist -libname WORK $REF_NETLIST
read_verilog -i -netlist -libname WORK $IMPL_NETLIST
set_constant scan_cg_en_i 0
set_constant scan_en 0
set_constant scan_in 0
match
verify

```

포인트:

DFT insertion은 구조를 크게 바꾼다. 그래서 post-DFT netlist를 STA/ATPG로 넘기기 전에 N2N Formality로 functional equivalence를 확인한다.

10. TetraMAX ATPG

파일:

```
4_ATPG/0_Script/run_tmax_stuck_at_topo.tcl
```

목적:

post-DFT scan netlist와 SPF를 읽어서 stuck-at fault pattern을 생성하고 coverage를 확인한다.

입력:

파일	의미
post-DFT .vg	scan cell이 들어간 gate netlist
.spf	scan chain/test protocol 정보

핵심:

```

read_netlist -library ...saed32nm.tv
read_netlist $NETLIST_FILE
set_rules B12 ignore
set_learning -atpg_equivalence
run_build_model $DESIGN_NAME

set_drc -allow_unstable_set_resets
set_drc -clock -dynamic -nodisturb_clock_grouping
run_drc $SPF_FILE

set_faults -model stuck
add_faults -all
set_atpg -capture_cycles 4 -abort_limit 32 -num_processes 4
set_atpg -fill adjacent -coverage 98
run_atpg

```

명령	의미
<code>read_netlist -library</code>	ATPG용 standard cell Verilog model 읽기
<code>run_build_model</code>	TetraMAX 내부 ATPG model 구성
<code>run_drc \$SPF_FILE</code>	scan chain/test protocol 검사
<code>set_faults -model stuck</code>	stuck-at fault model 사용
<code>add_faults -all</code>	전체 fault universe 생성
<code>run_atpg</code>	pattern 생성

주의:

B12 ignore, Z3 warning 같은 rule 완화는 first-pass ATPG 진행을 위한 분류다. signoff-clean claim이 아니다.

11. PrimeTime STA

파일:

```

6_STA/0_Script/run_pt_pre_dft_10ns.tcl
6_STA/0_Script/run_pt_pre_dft_10ns_sdf.tcl
6_STA/0_Script/run_pt_post_dft_10ns_sdf.tcl

```

역할:

script	역할
run_pt_pre_dft_10ns.tcl	일반 pre-DFT netlist wire-load STA
run_pt_pre_dft_10ns_sdf.tcl	topo pre-DFT netlist + SDF STA
run_pt_post_dft_10ns_sdf.tcl	post-DFT scan netlist + SDF STA

핵심 패턴:

```
set link_path [list * $RVT_TT_DB $LVT_TT_DB $HVT_TT_DB]
read_verilog $NETLIST
current_design $TOP_NAME
link_design
read_sdc $SDC_FILE
read_sdf $SDF_FILE
check_timing -verbose
report_global_timing
report_timing -delay_type max
report_timing -delay_type min
report_constraint -all_violators
report_analysis_coverage
report_annotated_delay
```

report	보는 것
check_timing	no_clock, unconstrained endpoint, generated clock 문제
report_global_timing	WNS/TNS 요약
report_timing -delay_type max	setup path
report_timing -delay_type min	hold path
report_constraint -all_violators	timing/electrical violation
report_analysis_coverage	STA coverage
report_annotated_delay	SDF annotation 적용 여부

SDF STA 의미:

```
netlist = 구조
SDC      = constraint
SDF      = delay number
```

PT에서 셋을 같이 읽어 DC timing report를 독립적으로 재검증한다.

12. 핵심 개념 정리

Functional mode vs Scan mode

Functional mode:

```
scan_en = 0
scan_cg_en_i = 0
scan_in = 0
```

Scan mode:

```
scan_en = 1
scan_cg_en_i = 1
scan chain active
```

FM/STA는 대부분 functional mode 기준이다. TetraMAX/DFT는 scan mode를 별도로 정의한다.

SVF

DC/DFT가 design을 어떻게 바꿨는지 Formality에게 알려주는 guide file

clock gating, hierarchy optimization, DFT insertion 이후 equivalence matching에 중요하다.

SDF

netlist에 delay number를 annotate하는 파일

PT SDF STA에서 DC topo delay 추정을 다시 확인한다.

SPF

scan chain과 test protocol 정보를 담는 파일

TetraMAX가 scan DRC/ATPG를 수행할 때 필요하다.

DDC vs VG

파일	성격	주 사용처
.ddc	Synopsys 내부 design DB	DC/DFT reload
.vg	Verilog gate netlist	FM/PT/TetraMAX/외부 handoff

13. 공부 순서

1. configs/library_setup.tcl
2. filelists/cv32e40p_dc.tcl
3. constraints/cv32e40p_func_10ns.sdc
4. 2_Synthesis/0_Script/run_analyze_elab_link.tcl
5. 2_Synthesis/0_Script/run_compile_10ns_topo.tcl
6. 2.5_FM_R2N/0_Script/run_fm_r2n_topo.tcl
7. 3_DFT/0_Script/run_insert_dft_10ns_topo.tcl
8. 5_FM_N2N/0_Script/run_fm_n2n_topo.tcl
9. 4_ATPG/0_Script/run_tmax_stuck_at_topo.tcl
10. 6_STA/0_Script/run_pt_pre_dft_10ns_sdf.tcl
11. 6_STA/0_Script/run_pt_post_dft_10ns_sdf.tcl

각 단계에서 볼 질문:

단계	질문
library	어떤 cell/corner로 mapping하나
filelist	RTL compile order가 왜 이 순서인가
SDC	어떤 mode와 clock을 검증하나
DC	RTL이 어떤 netlist/artifact로 변하나
FM R2N	합성 netlist가 RTL과 같은가
DFT	scan chain을 어떻게 정의하고 삽입하나
FM N2N	scan 삽입 후 functional equivalence가 유지되나
ATPG	scan chain으로 stuck-at fault를 얼마나 detect하나
PT	DC 결과를 독립 STA tool로 재검증했나

14. 면접 설명 문장

짧게:

CV32E40P 공개 RTL을 wrapper 기준으로 정리한 뒤, SAED32 TT mixed-VT library에서 DC Graphical synthesis를 수행했습니다. 합성 결과는 SVF 기반 Formality R2N으로 RTL 등가성을 확인했고, DFT Compiler로 muxed scan을 삽입한 뒤 post-DFT netlist를 N2N Formality로 다시 검증했습니다. 이후 TetraMAX stuck-at ATPG와 PrimeTime SDF STA로 scan 구조와 timing을 재확인했습니다.

기술적으로:

단순히 DC compile 결과만 본 것이 아니라, pre-DFT topo netlist, SDC, SDF, SVF를 후속 tool handoff artifact로 남겼습니다. R2N/N2N Formality는 scan/test port를 functional constant로 고정해 normal mode equivalence를 검증했고, DFT 단계에서는 scan clock/reset/test mode/scan path를 명시적으로 정의해 SPF를 생성했습니다. PrimeTime에서는 SDF annotation과 analysis coverage를 확인해 DC timing report와 독립적으로 setup/hold를 재검증했습니다.

15. 개선 포인트

개선	이유
PROJECT_ROOT 환경변수화	다른 machine/path에서 재현 가능하게 만들기 위함
library path 환경변수화	private path 노출 방지, repo template화
MCMM 확장	현재는 TT 단일 corner. SS/FF 및 setup/hold scenario 필요
backend handoff 정리	ICC2/OpenROAD로 넘길 netlist/SDC/library/floorplan 조건 필요
report parser 추가	WNS/TNS/area/power/coverage를 자동 table화

16. 안전한 Claim Boundary

가능:

CV32E40P front-end closure baseline
DC/DC Graphical synthesis
Formality R2N/N2N pass
DFT scan insertion
TetraMAX stuck-at ATPG
PrimeTime SDF STA

금지:

RTL-to-GDS 완료
post-route signoff 완료
IR/EM 완료
production DFT signoff
multi-corner signoff closure

현재 프로젝트는 **front-end closure**로 강하다. backend evidence가 생기면 그때 RTL-to-GDS로 올린다.